

Assignment Report: Fine-Tuning DistilBERT for Toxicity Detection

Aaron Modiyil Joseph
22018497

GitHub: github.com/aaronmj7/Assignment-3-LLM
University of Hertfordshire
15 August 2024

Introduction

In the vast digital landscape of social media, discussion forums, and comment sections, toxicity poisons conversations, drives away users, and harms online communities. As we navigate this interconnected world, identifying and mitigating toxic content becomes increasingly critical. This assignment delves into precisely this challenge and leverages Large Language Models (LLM) to predict whether a given comment is toxic or not. The task is a binary classification: given a comment, determine whether it exhibits toxic behaviour.

The motivation for developing an automated toxicity classifier is to help create safer virtual spaces and improve user experience. Toxic comments, ranging from hate speech to personal attacks, silence voices, intimidate, and drive away genuine contributors. Swiftly flagging toxicity will empower individuals and contribute to creating a more respectful digital environment.

1 Data Overview and Pre-Processing

The Wiki Toxic dataset serves as a modified and cleaned version of the original dataset used in the Kaggle Toxic Comment Classification Challenge (conducted during 2017/18). It comprises comments collected from Wikipedia forums and is labelled into two distinct categories: toxic and non-toxic. The dataset is obtained from Hugging Face, a reputable platform for sharing and accessing natural language processing (NLP) datasets and pre-trained models.

The dataset is well-suited for text classification tasks, particularly for training models to identify toxicity in sentences and classify them accordingly. All data instances in the dataset are in English language. Each data instance comprises the following components: `id`: A unique identifier string associated with each comment, `comment_text`: The actual textual content of the comment as a string, `label`: An integer label, 0 for non-toxic and 1 for toxic comments. The dataset is divided into several subsets: a `train` set containing 127,656 instances, a validation set containing 31,915, a `test` set containing 63,978 and a `balanced_train` set, a subset of the training data with 25,868 instances, balanced in terms of toxic and non-toxic comments.

Prior to commencing model training, we thoroughly preprocess the Wiki Toxic dataset. The `balanced_train` dataset is separated into training and validation subsets. Due to our restricted computational resources, we choose to use this smaller subset and its inherent balance simplifies our task. A validation size of 30% is used and the random seed is set up to guarantee reproducibility.

Leveraging the Hugging Face Transformers library, the tokenizer associated with our chosen model is utilized to convert raw comment text into tokenized input suitable for the model. The tokenizer is verified to be a fast tokenizer and its maximum token length is determined. A custom function, `tokenize` is defined, which also truncates comments to a maximum length of 128 tokens to run our model on GPUs found on Google Colab. `.map` method is used to tokenize the whole dataset with the `tokenize` function.

To ensure a consistent input format for training and validation, a data collator that handles padding is defined. Tokenized training and validation data is converted into TensorFlow datasets using `.to_tf_dataset` with the defined data collator and batch size of 16.

2 Model

BERT (Bidirectional Encoder Representations from Transformers) is a transformer-based model developed by Google that has revolutionized natural language processing (NLP). BERT is an encoder-only transformer model that has a bidirectional approach to understanding text, meaning it considers the context from both the left and right of each word simultaneously. This allows BERT to capture the nuanced meaning of words in context, leading to more accurate and robust language understanding. BERT is pre-trained on Google's BooksCorpus and Wikipedia using two tasks: Masked Language Modeling (MLM), where some words in the input are masked and the model predicts them, and Next Sentence Prediction (NSP), where the model predicts whether one sentence follows another [1].

DistilBERT is a smaller, faster, and more efficient version of BERT. It is created through a process called knowledge distillation, where a smaller model (the student) is trained to replicate the behaviour of a larger model (the teacher), in this case, BERT. The goal is to compress the large BERT model into a more lightweight version without significant loss in performance. DistilBERT has 6 layers compared to the 12 layers in BERT-base

making it 60% faster and use 40% fewer parameters while retaining 97% of performance. Considering the constraints of limited computational and time resources, DistilBERT stands out as the optimal choice for the current task [2].

The DistilBERT model is initialized using the `TFAutoModelForSequenceClassification` class for a binary classification task with randomly initialized weights for the new head suitable for sequence classification. An AdamW optimizer with a learning rate of $1e^{-6}$ and a weight decay rate of 0.01 is defined with `create_optimizer`. An extremely small learning rate is used to prevent the model from making large updates to the weights, which could disrupt the valuable pre-trained features. The Sparse Categorical Cross entropy is the loss function since it is appropriate for a classification task with integer labels. `from_logits` is set to `True` as the model outputs raw logits. The model is compiled with the defined optimizer, loss function and the accuracy metric. The model is trained on the train and validation TensorFlow datasets defined for only 3 epochs to avoid overfitting [3].

After completing the training, the model was saved to Google Drive for easy access during deployment. The deployment was handled using the `ipywidgets` library, which provided an interactive UI with a text box. When a user submits text through the text box and clicks the predict button, the input is first tokenized and then passed to the trained model for prediction. The model's output is in the form of logits, which is converted to probabilities and the most probable class is determined. The class and the probability are then returned.

3 Results and Discussion

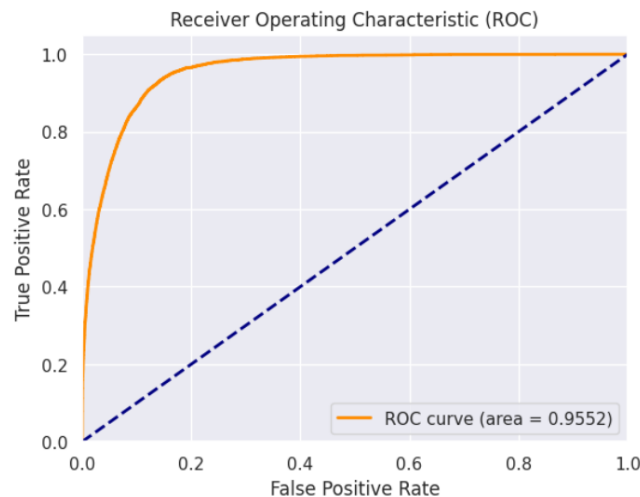


Figure 1: ROC Plot

The model achieved an accuracy of 92% during both training and validation, indicating that it learned the classification task well on balanced datasets. On the imbalanced test set, which contains fewer toxic comments, the model achieved a high Receiver Operating Characteristic - Area Under the Curve (ROC-AUC) score of 0.9552 (Figure 1). This indicates that the model has a high overall ability to distinguish between toxic and non-toxic comments.

Classification Report:					
	precision	recall	f1-score	support	
non	0.99	0.84	0.91	57735	
tox	0.39	0.94	0.56	6243	
accuracy			0.85	63978	
macro avg	0.69	0.89	0.73	63978	
weighted avg	0.93	0.85	0.88	63978	

Figure 2: Classification Report

The classification report (Figure 2) shows that the model performs exceptionally well in the non-toxic class,

with high precision. However, the recall for the non-toxic class is significantly lower, indicating that the model occasionally misclassifies non-toxic comments as toxic. In contrast, for the toxic class, the model has a low precision of 0.39, indicating that a significant number of non-toxic comments are incorrectly classified as toxic. This points to a higher rate of false positives. Despite this, the recall for the toxic class is notably high, at 0.94, indicating that the model is extremely effective at identifying the most toxic comments.

These results indicate that while the model performs well on balanced data, its generalization to imbalanced data is limited. The low precision for the toxic class on the imbalanced test set suggests a tendency for false positives, possibly due to the model being overly confident in its toxic predictions. Despite this, the high recall demonstrates the model's effectiveness in capturing toxic content.

In this context, the trade-off between precision and recall may be acceptable, as misclassifying non-toxic comments as toxic might be more tolerable than failing to identify actual toxic comments. However, this trade-off should be carefully considered, particularly in applications where the cost of false positives is high.

A variety of strategies can be used to improve the model's performance. Fine-tuning the model on a more representative, imbalanced dataset may improve its ability to generalize to real-world scenarios in which toxic comments are less common. This would improve the model's ability to handle skewed distributions and predict minority classes more accurately. Post-processing techniques, such as using a secondary model to filter out false positives, can also help to improve predictions and reduce errors in toxic comment classification.

However, the efficacy of these strategies is frequently hampered by factors such as insufficient data and resources. Having high-quality, annotated data is critical, especially when capturing the complexities of toxic comments, which can vary greatly in context and severity. It is critical to address the subjective nature of toxicity, which can vary depending on individual perspectives. In order to improve the model's overall accuracy and reduce misclassification, incorporating a diverse set of annotators, as well as taking context-aware models into account and expanding the dataset with more diverse examples, could aid in the development of a more reliable and flexible system and significantly improve the model's robustness. Furthermore, training and/or fine-tuning larger and more accurate models may produce better results, but it will require more computational and time resources.

Conclusion

The results indicate a strong overall performance, particularly reflected by the high ROC-AUC score, which suggests that the model can effectively distinguish between toxic and non-toxic comments. However, the disparity in precision and recall between the toxic and non-toxic classes highlights the challenges of applying the model to imbalanced real-world data. Specifically, the model demonstrates a tendency to over-predict toxicity, leading to a higher rate of false positives. To enhance the model's generalization capability, especially in the face of skewed data distributions, further fine-tuning on imbalanced datasets, as well as incorporating additional strategies like a secondary model to filter out false positives might be necessary. Additionally, training and/or fine-tuning larger and more accurate models may produce better results, but this approach will require more computational and time resources.

References

- [1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," 2019.
- [2] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter," 2020.
- [3] "Hugging face nlp course." Accessed on August 15, 2024.